

# j4\_talk

Audio and mouth LED firmware for the Johnny 4 robot. Runs on a Teensy 4.1 with an Audio Shield Rev D. Plays WAV files from SD card as a serial-controlled jukebox and drives 18 mouth LEDs that react to the audio amplitude in real time.

## What this board does

- Scans the SD card for WAV files at boot and serves them as a jukebox
- Plays, stops, and reports tracks over a serial link from the j4\_receiver (PLAY / STOP / LIST? in, PLAYING: out)
- Reports end of track over serial so the now-playing highlight clears on the controller display
- Sends a PING heartbeat once per second so j4\_receiver (and the controller) can show j4\_talk as connected on the status screen
- Applies the controller's volume pot to the SGTL5000 codec (previously fixed at 0.8)
- Analyzes live audio amplitude using the AudioAnalyzePeak library
- Drives 18 LEDs across 5 amplitude tiers -- the mouth opens progressively as audio gets louder
- PWM brightness on the leading tier is gamma-corrected so the ramp looks visually linear

## LED layout

18 PWM-capable Teensy pins drive 18 LEDs, one per pin, named after their physical position: **10 across the front** ( front\_LED\_01 on the far left up to front\_LED\_10 on the far right) and **8 down the side** ( side\_LED\_01 at the top down to side\_LED\_08 ).

**Front row, left to right:**

| Name | f01 | f02 | f03 | f04 | f05 | f06 | f07 | f08 | f09 | f10 |
|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Pin  | 2   | 3   | 4   | 5   | 6   | 9   | 16  | 17  | 22  | 14  |

**Side, top to bottom:**

| Name | s01 | s02 | s03 | s04 | s05 | s06 | s07 | s08 |
|------|-----|-----|-----|-----|-----|-----|-----|-----|
| Pin  | 15  | 28  | 29  | 33  | 36  | 37  | 10  | 11  |

Each tier of LEDs lights up as amplitude crosses its threshold, spreading outward from the middle of the front row:

| Tier | LEDs                       | Pins          | Description                            |
|------|----------------------------|---------------|----------------------------------------|
| 1    | front 05, 06               | 6, 9          | innermost front pair -- first to light |
| 2    | front 04, 07 + side 01, 02 | 5, 16, 15, 28 |                                        |
| 3    | front 03, 08 + side 03, 04 | 4, 17, 29, 33 |                                        |
| 4    | front 02, 09 + side 05, 06 | 3, 22, 36, 37 |                                        |
| 5    | front 01, 10 + side 07, 08 | 2, 14, 10, 11 | outermost (full open)                  |

Four channels were moved in v1\_8: pins 24/25 are Serial6 to the receiver (driving them as PWM killed the serial link), and pins 38/39 have no PWM hardware on the Teensy 4.1 (they could only snap on/off). Those four LED channels now live on 14/15 and 10/11.

# Hardware

| Component       | Details                                           |
|-----------------|---------------------------------------------------|
| Microcontroller | Teensy 4.1                                        |
| Audio           | Teensy Audio Shield Rev D (SGTL5000 codec)        |
| Storage         | microSD card (in Teensy 4.1 built-in slot)        |
| LED switching   | RFP30N06LE N-channel MOSFET (one per LED channel) |

The MAX98357A used in earlier versions is no longer part of this build.

## MOSFET wiring

Each of the 18 LED channels uses a RFP30N06LE N-channel MOSFET to switch a 12V LED supply from 3.3V Teensy logic. The RFP30N06LE is rated 30A / 60V and fully enhances at  $V_{gs} \sim 2V$ , so the 3.3V output drives it with margin to spare.

Repeat this wiring for each channel:

| MOSFET pin | Connection               |
|------------|--------------------------|
| Gate       | Teensy PWM pin           |
| Source     | GND (shared with Teensy) |
| Drain      | LED cathode (-)          |

The LED anode (+) connects to 12V through a current-limiting resistor sized for the LED forward voltage and desired current.

## Amplitude tiers

The amplitude-to-tier mapping uses piecewise linear thresholds defined in `THRESHOLDS[ ]`. Tune those five values to match your audio levels. Uncomment the `Serial.print` lines in `updateLEDs()` to watch the smoothed peak and level values in real time while tuning.

The smoothing constant `PEAK_SMOOTHING` (default 0.80) controls how fast the display reacts. Lower values respond faster but look noisier. Higher values smooth out transients but feel sluggish.

## Pin assignments

| Teensy Pin | Function                         |
|------------|----------------------------------|
| 2          | front_LED_01 (tier 5)            |
| 3          | front_LED_02 (tier 4)            |
| 4          | front_LED_03 (tier 3)            |
| 5          | front_LED_04 (tier 2)            |
| 6          | front_LED_05 (tier 1, innermost) |
| 9          | front_LED_06 (tier 1, innermost) |

|    |                                                |
|----|------------------------------------------------|
| 10 | side_LED_07 (tier 5)                           |
| 11 | side_LED_08 (tier 5)                           |
| 14 | front_LED_10 (tier 5)                          |
| 15 | side_LED_01 (tier 2)                           |
| 16 | front_LED_07 (tier 2)                          |
| 17 | front_LED_08 (tier 3)                          |
| 22 | front_LED_09 (tier 4)                          |
| 24 | Serial6 TX to j4_receiver (receiver GPIO 2)    |
| 25 | Serial6 RX from j4_receiver (receiver GPIO 17) |
| 28 | side_LED_02 (tier 2)                           |
| 29 | side_LED_03 (tier 3)                           |
| 33 | side_LED_04 (tier 3)                           |
| 36 | side_LED_05 (tier 4)                           |
| 37 | side_LED_06 (tier 4)                           |

Audio Shield Rev D occupies 7, 8, 20, 21, 23 (I2S) and 18, 19 (I2C); the SD card uses the Teensy 4.1 built-in slot. All LED pins are chosen to avoid those.

## Pin diagram

Standard Teensy 4.1 layout, component side up, USB at the TOP. The two long edges read top-to-bottom; \* marks a pin this firmware uses.

```

+===== [ USB ] =====+
      GND  -|  GND          5V  |-  5V
          0  -|  0          GND  |-  GND
          1  -|  1        3.3V  |-  3.3V
front_LED_01 (T5) 2 *-| 2        23  |-  (audio I2S)
front_LED_02 (T4) 3 *-| 3        22  |-* front_LED_09 (T4)
front_LED_03 (T3) 4 *-| 4        21  |-  (audio I2S)
front_LED_04 (T2) 5 *-| 5        20  |-  (audio I2S)
front_LED_05 (T1) 6 *-| 6        19  |-  (audio I2C)
(audio)          7  -| 7        18  |-  (audio I2C)
(audio)          8  -| 8        17  |-* front_LED_08 (T3)
front_LED_06 (T1) 9 *-| 9        16  |-* front_LED_07 (T2)
side_LED_07 (T5) 10 *-| 10       15  |-* side_LED_01 (T2)
side_LED_08 (T5) 11 *-| 11       14  |-* front_LED_10 (T5)
          12  -| 12       13  |-  (LED_BUILTIN)
Serial6 TX      24 *-| 24       41  |-
Serial6 RX      25 *-| 25       40  |-
          26  -| 26       39  |-
          27  -| 27       38  |-
side_LED_02 (T2) 28 *-| 28       37  |-* side_LED_06 (T4)

```

```

side_LED_03 (T3) 29 *-| 29          36 |-* side_LED_05 (T4)
                 30 -| 30          35  |-
                 31 -| 31          34  |-
                 32 -| 32          (bottom: SD card + SMD pads)
side_LED_04 (T3) 33 *-| 33          |
                 +=====+

```

```

Serial6: 24 (TX) -> j4_receiver GPIO 2 ; 25 (RX) <- j4_receiver GPIO 17
Audio Shield Rev D uses 7, 8, 20, 21, 23 (I2S) + 18, 19 (I2C) + built-in SD.
front_LED_01..10 = front row left to right (pins 2,3,4,5,6,9,16,17,22,14);
front 05/06 light first. side_LED_01..08 = down the side, top to bottom
(pins 15,28,29,33,36,37,10,11).

```

## Jukebox serial protocol

The board talks to j4\_receiver over Serial6 at 115200 baud:

| Direction | Message                                           | Meaning                                              |
|-----------|---------------------------------------------------|------------------------------------------------------|
| in        | volume,neck                                       | CSV of volume (0-100) and neck value each frame      |
| in        | PLAY:<id>                                         | start the track with that two-character ID           |
| in        | STOP                                              | stop playback                                        |
| in        | LIST?                                             | re-send the file list                                |
| out       | PLAYING:<id>                                      | a track started (empty payload = stopped / finished) |
| out       | LIST_START:<n> ... <id> <name> ... LIST_END       | the SD-card file list                                |
| in        | FACES?                                            | dump every saved face preset                         |
| in        | FACESAVE:<key>,<11 values>,<toggles>              | save/overwrite one face and rewrite FACES.TXT        |
| out       | FACE:<key>,<11 values>,<toggles> ... FACE_END:<n> | the saved-face dump                                  |
| out       | FACEOK:<key> / FACEERR:<key>                      | SD write confirmed / failed                          |

## Face presets (FACES.TXT)

The controller's right keypad saves and recalls facial expressions ("faces"); this board is their persistent home because it has the only microSD in the system. FACES.TXT in the SD root holds one CSV line per face: <key>,<11 values>,<toggles> -- the keypad character, the 11 face-pot values (iris, color, brightness, eyebrows, nose, eyelids; the controller owns the meaning and order), and a LASER/VENT/EYE POP toggle bitmask. The whole file is rewritten on every save (it is at most 16 short lines). A missing or unreadable FACES.TXT simply means zero saved faces; every malformed line is skipped field-by-field, and boot never hangs on any of it.

## WAV files

Place WAV files in the root of the SD card. The firmware scans them at boot and plays a file when it receives a matching PLAY:<id> command. 16-bit PCM, 44100 Hz stereo is the standard format for the Teensy Audio library.

## Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output
pio device monitor
```

## Related projects

- [j4 controller](#) -- the TTGO T-Display transmitter that sends control data
- [j4 receiver](#) -- the ESP32 receiver board on the robot that feeds this board over serial